

Project PIV 2025

Page • Tag • 1 backlink

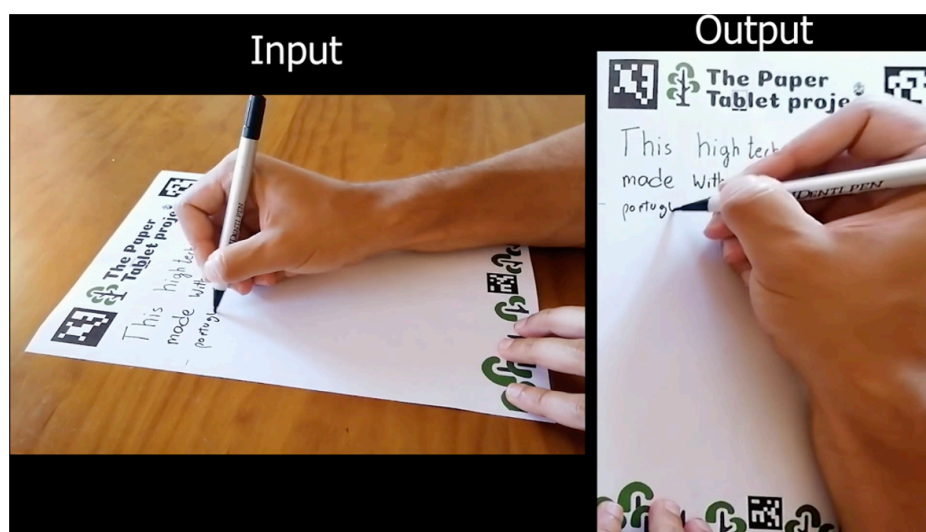
The Paper Tablet Project

Motivation

A moving camera observes a person filling out or manipulating a paper form placed on a table. As the camera or the paper moves, the apparent geometry of the document changes due to perspective distortions.

To enable automatic analysis, digitization, or interaction (e.g., handwriting capture), the system must continuously track the document and produce a **perspective-corrected (frontal) view** of the form in each frame.

In the figure below we illustrate part of the intended output of the project.



However we will go one step further and use 3D information to better render images of the document.

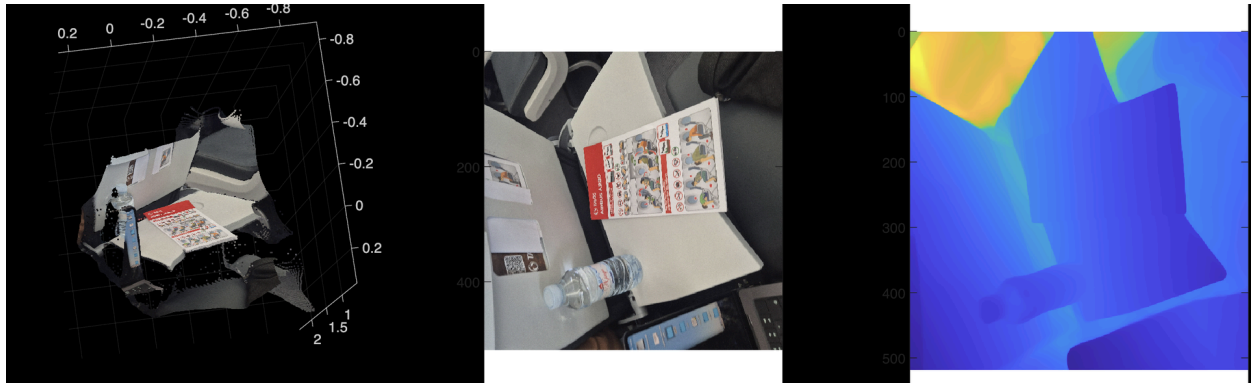
A project in 3 steps

The project develops mainly in two parts plus one optional

Part I - The input is a sequence of images and one template (one image). The input sequence may have images showing the template or not. The camera may drift away from the template but your code must provide a transformation from every image to the template.

Part II - We add depth to the RGB image. The sequence will include both the RGB image and a depth image as captured by a depth camera. This way we will be able

to detect and register objects of any shape and scenes not necessarily planar. In short, this part is intended to use 3D data to its largest scope.



As the figure shows, with depth images a 3D point cloud can be computed which is a completely different representation with different properties that you will learn during the course.

Part III - Time allowing, the use the 2D/3D data and any extra information you may propose to use (using ready-made systems) you may be able to build an interesting/useful/sophisticated application. Examples could be a hand tracker to remove it from the output or use hand position and action to create an interface system using a paper form as "buttons".

Part I – Homography Estimation from RGB Images

Objective: Develop a robust homography estimation pipeline using only RGB data.

Typical Implementation Steps

1. Feature Detection and Matching

- Detect local features using a scale- and rotation-invariant detector such as **SIFT**, **SURF**, or **ORB**.
- Compute descriptors and establish correspondences between the current frame and a **reference image** of the form.

2. Outlier Removal with RANSAC

- Many feature matches may be incorrect due to occlusions, texture ambiguities, or lighting changes.
- Use **Random Sample Consensus (RANSAC)** to robustly estimate a homography by repeatedly sampling minimal sets of four correspondences and selecting the model with the largest inlier consensus.

3. Homography Estimation via Least Squares

- Once inliers are identified, refine the homography parameters using **linear least squares (LSE)** or the **Direct Linear Transform (DLT)** algorithm, followed by normalization.
 - Apply the homography to warp the current frame, generating a **rectified, perspective-corrected image** of the document.
 - Using SGD (or Newton-like) compute transformation as a nonlinear-optimization problem. (very optional)
-

Part II – Incorporating Depth Information (RGB-D Extension)

Objective

Enhance the system with an **RGB-D camera** that provides a registered RGB image and depth map at each time instant, allowing 3D geometry-based tracking and object detection.

Implementation Steps

1. 3D Point Extraction and Registration

- Using camera intrinsics, convert 2D feature correspondences into **3D point clouds**.
- Estimate the rigid 3D transformation (rotation + translation) between frames using the **Procrustes method** (orthogonal Procrustes analysis) to minimize alignment error between corresponding 3D points.

2. Robust Estimation with RANSAC

- Integrate **RANSAC** to reject outlier 3D correspondences that violate the planar or rigid transformation model.
- Improves robustness against mismatched features or moving elements (e.g., hands, pen).

3. Plane and Object Segmentation

- Fit a dominant plane to the 3D points (e.g., via RANSAC plane fitting) to identify the document surface.
 - Points that lie significantly above the plane can be labeled as **out-of-plane objects**, allowing **hand or pen detection** and masking.
-

Expected Outcome

A two-stage system capable of:

- Tracking and rectifying a planar document in real time under camera motion (**Part I**).
- Leveraging depth information to improve registration accuracy and detect non-planar elements (**Part II**).

The final pipeline demonstrates how **robust 2D and 3D estimation techniques**—including **RANSAC**, **LSE**, and **Procrustes analysis**—can be combined for reliable geometric vision tasks.

FINAL DEFINITIONS FOR CODE AND SUBMISSION

Part 1

your code must include two files:

one named main1.py (main.m)

one named part1.py (part1.m)

The running command:

```
python main1.py path_to_refimg path_images_dir path_feature_dir
path_output_dir
```

path_to_refimg: the path for the image file with the template

path_images_dir : path to the directory with the image sequence (jpg files with file names somename_NNNN.jpg where NNNN is the sequential number of the images)

path_feature_dir : path to the directory where image features are stored in .mat format somename_NNNN.mat

path_output_dir: path to the output results.

main1.py :

- may import cv2
- must import part1.m
- tasked with processing image files and generate files with features, stored in a special directory.
- The format of each file is defined by the script we distributed before <https://github.com/pivist/features/blob/main/main/main.py>. The easy way is to include that code in main1.py.

- should call function `part1(args)` - defined next

`part1.py`

- Can not import `cv2` or any package other than the allowed packages (`flann` or `replacement`, `numpy`, `scipy`). Special useful packages may be requested with justification.
- Must have one function called `part1`:
 - `part1(path1, path2, path3, path4)` where the arguments are the inputs of the run commands above.
- Must save in `path4` (`path_output_dir`) files `homography_NNNN.mat` with a dictionary `{"H":H}` where `H` is a (3,3) numpy matrix representing the homography from frame `NNNN` to the reference image.

Part 2:

Here the input data is a reference rgb and depth image, a sequence of images and depth maps and a list of camera intrinsics. The output is the rigid transformation from each image to the template.

Your code must include one file named `main2.py` (`main2.m`)

The running command:

```
python main2.py path_to_refimgdir path_images_dir path_output_dir
```

`path_to_refimgdir`: the path to the image/depth DIRECTORY with the template data: `templatergb.jpg`, `templatedepth.mat` and `cam.mat`

`path_images_dir` : path to the directory with the image and depth sequence (rgb files with file names `somename_NNNN.jpg` and `somename_NNNN.mat` and where `NNNN` is the sequential number of the images

`path_output_dir`: path to the output results.

depth images are .mat files with a dictionary `{"depth":depth,"K":K}` where `depth` is a depth image (numpy matrix) and `K` is a (3,3) matrix with the intrinsics.

`main2.py` :

- may import `cv2` or other packages but not packages that compute 3D transformations or outlier removal. In principle you should only include `cv2` and the other from `part1`

- Must save in path_output_dior files transform_NNNN.mat with a dictionary {"R":R,"T":T} where R is a (3,3) numpy matrix representing the rotation from frame NNNN to the reference image and T is a (3,1) numpy vector with the translation.
-

Report

The Short report (6 pages max)

Should have sections Part 1, Part 2 and optionally part 3. In each section describe:

- Algorithmic details (feature choice, RANSAC parameters, numerical methods).
 - Experimental setup and evaluation.
 - Example results illustrating the outcome
-

SUBMISSION

Code must be submitted in two directories named part1 and part2 in the PIV server by December 22, 23:59